Universidad Interamericana de Panamá

Facultad de ingeniería, Arquitectura y Diseño

Lic. En Sistemas Computacionales Con Énfasis En Desarrollo De Sistemas Avanzados En Redes Y Software.

Base De Datos II

Prof. Maryon Torres

Charla #2

Cristian Diaz PN-0048541

Roberto Lasso 8-1034-727

Carlos Poveda 8-976-1952

Simulación de Sistema Producer–Consumer tipo Kafka

Introducción

Este proyecto consiste en una simulación simplificada de un sistema de mensajería basado en el modelo Producer–Consumer, inspirado en herramientas reales como Apache Kafka.

El objetivo principal es entender cómo se generan, transmiten y procesan datos en tiempo real dentro de sistemas distribuidos.

Arquitectura del Sistema

El sistema está compuesto por tres elementos principales:

1. Productor (Producer)
2. "Tópico" (Simulado con archivo JSON)
3. Consumidor (Consumer)

Flujo de datos

[Productor] → [Archivo JSON (tópico)] → [Consumidor]

Componentes del Código

1. Productor (simulate_producer)

Función

Genera pedidos de forma automática y los envía al sistema.

Funcionamiento

- Selecciona un producto aleatorio
- Genera una cantidad aleatoria
- Asigna un ID incremental
- Agrega la hora actual

- Convierte el pedido a formato JSON
- Lo guarda en un archivo (simulando un tópico)

Estructura del pedido

```
{
  "id": 1,
  "producto": "Laptop",
  "cantidad": 2,
  "timestamp": "14:32:10"
}
```

Ejecución continua

El productor corre en un bucle infinito generando pedidos cada 1 a 3 segundos.

2. Tópico (Archivo JSON)

Función

Actúa como intermediario entre el productor y el consumidor.

Funcionamiento

- Se utiliza el archivo: kafka_topic_pedidos.json
- Cada línea representa un mensaje independiente
- Permite lectura secuencial (como un log)

Importante

En un sistema real, esto sería manejado por un broker como Kafka.

3. Consumidor (simulate_consumer)

Función

Lee y procesa los pedidos generados por el productor.

Funcionamiento

- Verifica si el archivo existe
- Lee nuevas líneas desde la última posición
- Convierte cada línea JSON en diccionario
- Imprime el pedido recibido
- Simula procesamiento (inventario y facturación)

Ejecución continua

El consumidor está constantemente verificando nuevos datos en tiempo real.

Librerías Utilizadas

time

- Manejo de pausas
- Generación de timestamps

json

- Conversión entre diccionarios y texto JSON

os

- Verificación y eliminación de archivos

random

- Generación de datos aleatorios

Simulación vs Sistema Real

Elemento	Simulación	Sistema Real (Kafka)
Tópico	Archivo JSON	Broker Kafka
Productor	Script Python	Aplicación conectada a Kafka
Consumidor	Script Python	Servicios suscritos
Mensajes	Líneas en archivo	Eventos en streaming

Ventajas del Modelo

- Procesamiento en tiempo real
- Desacoplamiento entre sistemas
- Escalabilidad
- Flexibilidad

Limitaciones de la Simulación

- No hay concurrencia real
- No existe tolerancia a fallos
- No hay particiones ni replicación
- El archivo puede crecer indefinidamente

Aplicaciones Reales

Este tipo de arquitectura se usa en:

- Sistemas de comercio electrónico
- Procesamiento de pagos
- Monitoreo de eventos
- Sistemas de logs

- Streaming de datos

Conclusión

Esta simulación permite comprender de forma clara el modelo Producer–Consumer aplicado al procesamiento de datos en tiempo real. A través de una implementación sencilla, se observa cómo se generan, transmiten y consumen los datos, así como el desacoplamiento entre los componentes del sistema.

El uso de un archivo JSON como intermediario facilita la comprensión del concepto de tópico, aunque de manera simplificada. Esto ayuda a visualizar cómo funcionan sistemas reales como Kafka, donde múltiples productores y consumidores interactúan de forma eficiente.

Si bien la simulación no incluye características avanzadas como concurrencia, tolerancia a fallos o distribución, cumple adecuadamente su propósito didáctico. En conjunto, este proyecto sienta una base sólida para avanzar hacia herramientas profesionales y comprender arquitecturas de streaming de datos en entornos reales.